You can see bindings all over the scope. Look down at your feet, and you see a bunch of local variables. Raise your head, and you see that you're standing within an object, with its own methods and instance variables; that's the current object, also known as self. Farther away, you see the tree of constants so clear that you could mark your current position on a map. Squint your eyes, and you can even see a bunch of global variables off in the distance.

But what happens when you get tired of the scenery and decide to move on?

Changing Scope

This example shows how scope changes as your program runs, tracking the names of bindings with the Kernel#local_variables method:

blocks/scopes.rb

```
v1 = 1
class MyClass
  v2 = 2
  local_variables    # => [:v2]
  def my_method
    v3 = 3
    local_variables
  end
  local_variables    # => [:v2]
end

obj = MyClass.new
obj.my_method        # => [:v3]
obj.my_method        # => [:v3]
local_variables      # => [:v1, :obj]
```

Track the program as it moves through scopes. It starts within the top-level scope that you read about in *The Top Level*, on page 35. After defining v1 in the top-level scope, the program enters the scope of MyClass's definition. What happens then?

Some languages, such as Java and C#, allow "inner scopes" to see variables from "outer scopes." That kind of nested visibility doesn't happen in Ruby, where scopes are sharply separated: as soon as you enter a new scope, the previous bindings are replaced by a new set of bindings. This means that when the program enters MyClass, v1 "falls out of scope" and is no longer visible.

In the scope of the definition of MyClass, the program defines v2 and a method. The code in the method isn't executed yet, so the program never opens a new scope until the end of the class definition. There, the scope opened with the class keyword is closed, and the program gets back to the top-level scope.